

Project structure

```
streamlit-ml-app/
├── app.py           # Main application
├── requirements.txt  # Dependencies
├── README.md        # Project documentation
├── .gitignore       # Git ignore file
├── .streamlit/
│   └── config.toml  # Streamlit configuration
├── src/
│   ├── __init__.py
│   ├── eda.py       # EDA functions
│   ├── model.py     # Model operations
│   ├── utils.py     # Utility functions
│   └── visualize.py  # Visualization functions
├── data/
│   └── sample_data.csv # Sample dataset
├── models/
│   └── model.pkl     # Trained model (optional)
└── notebooks/
    └── train_model.ipynb # Model training notebook
```

Main application file:

```
"""
```

Streamlit ML Application - Complete EDA & Prediction Platform

Author: Your Name

GitHub: <https://github.com/yourusername/streamlit-ml-app>

```
"""
```

```
import streamlit as st
```

```
import pandas as pd
```

```
import os
```

```
from pathlib import Path
```

```
# Import custom modules
```

```
from src.eda import show_eda
```

```
from src.model import load_model, make_prediction, train_model
```

```
from src.utils import load_data, get_available_models
```

```
from src.visualize import plot_feature_importance
```

```
# Page configuration
```

```
st.set_page_config(
```

```
    page_title="ML Analytics Platform",
```

```
    page_icon="🤖",
```

```
    layout="wide",
    initial_sidebar_state="expanded"
)
```

```
# Custom CSS for better UI
```

```
st.markdown("""
<style>
.main-header {
    font-size: 2.5rem;
    color: #1f77b4;
    text-align: center;
    margin-bottom: 2rem;
}
.info-box {
    background-color: #f0f2f6;
    padding: 1rem;
    border-radius: 0.5rem;
    margin: 1rem 0;
}
</style>
""", unsafe_allow_html=True)
```

```
# Initialize session state
```

```
if 'df' not in st.session_state:
    st.session_state.df = None
if 'model' not in st.session_state:
    st.session_state.model = None
if 'predictions' not in st.session_state:
    st.session_state.predictions = None
```

```
# Sidebar
```

```
with st.sidebar:
```

```
    st.image("https://streamlit.io/images/brand/streamlit-mark-color.svg", width=100)
    st.title("📊 ML Analytics")
    st.markdown("---")
```

```
# Navigation
```

```
page = st.radio(
    "📌 Navigation",
    ["🏠 Home", "📊 Exploratory Data Analysis", "🎯 Model Prediction", "⚙️ Model Training"]
)
```

```
st.markdown("---")
st.caption("Made with ❤️ using Streamlit")
```

```
st.caption(f"Version: 1.0.0")
```

```
# Main content
```

```
if page == "🏠 Home":
```

```
    st.markdown('<h1 class="main-header">🚀 ML Analytics Platform</h1>',  
unsafe_allow_html=True)
```

```
col1, col2, col3 = st.columns(3)
```

```
with col1:
```

```
    st.metric("📊 EDA Tools", "5+", "Visualizations")
```

```
with col2:
```

```
    st.metric("🤖 Models", "3", "Ready to use")
```

```
with col3:
```

```
    st.metric("⚡ Speed", "Fast", "Real-time predictions")
```

```
st.markdown("---")
```

```
# File uploader
```

```
st.subheader("📁 Upload Your Dataset")
```

```
uploaded_file = st.file_uploader(  
    "Choose a CSV file",  
    type=['csv'],  
    help="Upload a CSV file with your dataset"  
)
```

```
if uploaded_file is not None:
```

```
    try:
```

```
        df = pd.read_csv(uploaded_file)
```

```
        st.session_state.df = df
```

```
        st.success(f"✅ Dataset loaded successfully! Shape: {df.shape}")
```

```
st.subheader("📋 Data Preview")
```

```
st.dataframe(df.head(10), use_container_width=True)
```

```
# Quick stats
```

```
col1, col2, col3 = st.columns(3)
```

```
with col1:
```

```
    st.info(f"📈 **Rows:** {df.shape[0]}")
```

```
with col2:
```

```
    st.info(f"📈 **Columns:** {df.shape[1]}")
```

```
with col3:
```

```
    missing = df.isnull().sum().sum()
```

```
    st.info(f"📈 **Missing Values:** {missing}")
```

```

except Exception as e:
    st.error(f"❌ Error loading file: {str(e)}")
else:
    # Use sample data
    if st.button("📁 Load Sample Dataset"):
        sample_data = Path("data/sample_data.csv")
        if sample_data.exists():
            df = pd.read_csv(sample_data)
            st.session_state.df = df
            st.success("✅ Sample dataset loaded!")
            st.dataframe(df.head(10), use_container_width=True)
        else:
            st.warning("⚠️ Sample data not found. Please upload your own CSV file.")

    st.markdown("""
    <div class="info-box">
    <b>📝 Instructions:</b><br>
    1. Upload a CSV file using the uploader above<br>
    2. Explore your data with EDA tools<br>
    3. Train models and make predictions
    </div>
    """, unsafe_allow_html=True)

elif page == "📊 Exploratory Data Analysis":
    if st.session_state.df is not None:
        show_eda(st.session_state.df)
    else:
        st.warning("⚠️ Please upload a dataset first (go to Home page)")

elif page == "🎯 Model Prediction":
    if st.session_state.df is not None:
        st.subheader("🎯 Make Predictions")

        # Load or train model
        if st.session_state.model is None:
            model_path = Path("models/model.pkl")
            if model_path.exists():
                st.session_state.model = load_model(str(model_path))
                st.success("✅ Model loaded successfully!")
            else:
                st.warning("⚠️ No trained model found. Please train a model first.")

        if st.session_state.model is not None:
            # Get feature columns (exclude target if exists)

```

```

df = st.session_state.df
feature_cols = [col for col in df.columns if col != 'target']

st.subheader("🔧 Input Features")

# Create input fields dynamically
input_data = {}
cols = st.columns(3)
for idx, col in enumerate(feature_cols[:6]): # Limit to 6 features
    with cols[idx % 3]:
        input_data[col] = st.number_input(
            f"{col}",
            value=float(df[col].mean()),
            step=0.1,
            key=f"input_{col}"
        )

if st.button("🚀 Predict", type="primary"):
    features = [input_data[col] for col in feature_cols[:6]]
    prediction, probability = make_prediction(
        st.session_state.model,
        [features]
    )

    if prediction is not None:
        col1, col2 = st.columns(2)
        with col1:
            st.success(f"***Prediction:** {prediction[0]}")
        with col2:
            if probability is not None:
                st.metric("Confidence", f"{probability.max():.2%}")

    # Store predictions
    st.session_state.predictions = prediction
else:
    st.warning("⚠️ Please upload a dataset first (go to Home page)")

elif page == "⚙️ Model Training":
    st.subheader("⚙️ Train New Model")

    if st.session_state.df is not None:
        df = st.session_state.df

        # Model selection

```

```
model_type = st.selectbox(
    "Select Model",
    ["Random Forest", "Logistic Regression", "Decision Tree", "XGBoost"]
)
```

```
# Target column selection
target_col = st.selectbox(
    "Select Target Column",
    df.columns.tolist()
)
```

```
# Training parameters
st.subheader("📊 Training Parameters")
test_size = st.slider("Test Set Size", 0.1, 0.4, 0.2, 0.05)
random_state = st.number_input("Random State", value=42, step=1)
```

```
if st.button("🚀 Train Model", type="primary"):
    with st.spinner("Training model..."):
        try:
```

```
            model, metrics = train_model(
                df,
                target_col,
                model_type,
                test_size,
                random_state
            )
```

```
        # Save model
        model_path = Path("models/model.pkl")
        model_path.parent.mkdir(exist_ok=True)
        import joblib
        joblib.dump(model, model_path)
        st.session_state.model = model
```

```
        st.success("✅ Model trained and saved successfully!")
```

```
# Show metrics
col1, col2, col3 = st.columns(3)
with col1:
    st.metric("Accuracy", f"{metrics['accuracy']:.2%}")
with col2:
    st.metric("Precision", f"{metrics['precision']:.2%}")
with col3:
    st.metric("Recall", f"{metrics['recall']:.2%}")
```

```

        # Feature importance
        st.subheader("📊 Feature Importance")
        if hasattr(model, 'feature_importances_'):
            fig = plot_feature_importance(model, df.columns[:-1])
            st.pyplot(fig)

    except Exception as e:
        st.error(f"❌ Error training model: {str(e)}")
else:
    st.warning("⚠️ Please upload a dataset first (go to Home page)")

```

Source code file :

Package initialization

"""

Exploratory Data Analysis Module

"""

```

import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go

```

```

def show_eda(df):
    """Main EDA function"""
    st.title("📊 Exploratory Data Analysis")

    # Tabs for different EDA sections
    tab1, tab2, tab3, tab4, tab5 = st.tabs([
        "📄 Summary", "📈 Distributions", "🔗 Correlations",
        "📉 Trends", "🚨 Outliers"
    ])

    with tab1:
        show_summary(df)

    with tab2:
        show_distributions(df)

    with tab3:
        show_correlations(df)

```

```
with tab4:
    show_trends(df)
```

```
with tab5:
    show_outliers(df)
```

```
def show_summary(df):
    """Data summary"""
    col1, col2 = st.columns(2)
```

```
    with col1:
        st.subheader("📊 Dataset Overview")
        st.write(f"***Rows:** {df.shape[0]}")
        st.write(f"***Columns:** {df.shape[1]}")
        st.write(f"***Memory Usage:** {df.memory_usage().sum() / 1024**2:.2f} MB")
```

```
    with col2:
        st.subheader("🔍 Missing Values")
        missing = df.isnull().sum()
        if missing.sum() > 0:
            missing_df = pd.DataFrame({
                'Column': missing.index,
                'Missing': missing.values,
                'Percentage': (missing.values / len(df) * 100).round(2)
            })
            missing_df = missing_df[missing_df['Missing'] > 0]
            st.dataframe(missing_df, use_container_width=True)
        else:
            st.success("✅ No missing values found!")
```

```
    st.subheader("📋 Data Types")
    dtype_df = pd.DataFrame({
        'Column': df.dtypes.index,
        'Data Type': df.dtypes.values.astype(str)
    })
    st.dataframe(dtype_df, use_container_width=True)
```

```
    st.subheader("📈 Summary Statistics")
    st.dataframe(df.describe(), use_container_width=True)
```

```
def show_distributions(df):
    """Show feature distributions"""
    num_cols = df.select_dtypes(include=['float64', 'int64']).columns.tolist()
```



```

if num_cols:
    col = st.selectbox("Select feature", num_cols)

    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

    # Histogram
    sns.histplot(df[col], kde=True, ax=axes[0], color='steelblue')
    axes[0].set_title(f"Distribution of {col}")

    # Box plot
    sns.boxplot(x=df[col], ax=axes[1], color='lightcoral')
    axes[1].set_title(f"Boxplot of {col}")

    st.pyplot(fig)

    # Skewness and Kurtosis
    skew = df[col].skew()
    kurt = df[col].kurtosis()
    col1, col2 = st.columns(2)
    col1.metric("Skewness", f"{skew:.3f}")
    col2.metric("Kurtosis", f"{kurt:.3f}")

def show_correlations(df):
    """Show correlation matrix"""
    num_cols = df.select_dtypes(include=['float64', 'int64']).columns

    if len(num_cols) > 1:
        corr = df[num_cols].corr()

        fig, ax = plt.subplots(figsize=(10, 8))
        sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',
                    center=0, square=True, ax=ax)
        ax.set_title("Correlation Matrix")
        st.pyplot(fig)

        # Top correlations
        corr_pairs = corr.unstack().sort_values(ascending=False)
        corr_pairs = corr_pairs[corr_pairs < 1].head(10)
        st.subheader("📈 Top Correlations")
        st.dataframe(pd.DataFrame({
            'Pair': corr_pairs.index.map(str),
            'Correlation': corr_pairs.values
        }), use_container_width=True)

```

```
else:
    st.info("Need at least 2 numerical columns for correlation analysis")
```

```
def show_trends(df):
```

```
    """Show trends and relationships"""
```

```
    num_cols = df.select_dtypes(include=['float64', 'int64']).columns
```

```
    if len(num_cols) >= 2:
```

```
        col1 = st.selectbox("X-axis", num_cols, key='x')
```

```
        col2 = st.selectbox("Y-axis", [c for c in num_cols if c != col1], key='y')
```

```
        # Interactive scatter plot
```

```
        fig = px.scatter(df, x=col1, y=col2,
```

```
                        title=f"{col1} vs {col2}",
```

```
                        opacity=0.6,
```

```
                        trendline="ols")
```

```
        st.plotly_chart(fig, use_container_width=True)
```

```
def show_outliers(df):
```

```
    """Detect and visualize outliers"""
```

```
    num_cols = df.select_dtypes(include=['float64', 'int64']).columns
```

```
    if num_cols:
```

```
        col = st.selectbox("Select feature for outlier detection", num_cols)
```

```
        # IQR method
```

```
        Q1 = df[col].quantile(0.25)
```

```
        Q3 = df[col].quantile(0.75)
```

```
        IQR = Q3 - Q1
```

```
        lower_bound = Q1 - 1.5 * IQR
```

```
        upper_bound = Q3 + 1.5 * IQR
```

```
        outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
```

```
        st.write(f"***Number of outliers:** {len(outliers)}")
```

```
        st.write(f"***Percentage:** {len(outliers) / len(df) * 100:.2f}%")
```

```
        # Box plot
```

```
        fig, ax = plt.subplots(figsize=(10, 4))
```

```
        sns.boxplot(x=df[col], ax=ax, color='lightcoral')
```

```
        ax.axvline(lower_bound, color='green', linestyle='--', label='Lower Bound')
```

```
        ax.axvline(upper_bound, color='red', linestyle='--', label='Upper Bound')
```

```
        ax.legend()
```

```
        st.pyplot(fig)
```

Src/[model.py](#)

```
"""
```

Machine Learning Model Module

```
"""
```

```
import joblib
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import LabelEncoder
```

```
def load_model(model_path):
```

```
    """Load trained model"""
```

```
    try:
```

```
        model = joblib.load(model_path)
```

```
        return model
```

```
    except Exception as e:
```

```
        print(f"Error loading model: {e}")
```

```
        return None
```

```
def make_prediction(model, features):
```

```
    """
```

Make predictions with model

Args:

model: Trained model

features: 2D array of features

Returns:

predictions: Array of predictions

probabilities: Probability scores (if available)

```
    """
```

```
    try:
```

```
        features = np.array(features)
```

```
        predictions = model.predict(features)
```

```

    if hasattr(model, 'predict_proba'):
        probabilities = model.predict_proba(features)
        return predictions, probabilities
    return predictions, None
except Exception as e:
    print(f"Error making prediction: {e}")
    return None, None

```

```
def train_model(df, target_col, model_type, test_size=0.2, random_state=42):
```

```
    """
```

Train a machine learning model

Args:

df: DataFrame with features and target

target_col: Name of target column

model_type: Type of model to train

test_size: Test set proportion

random_state: Random seed

Returns:

model: Trained model

metrics: Dictionary of performance metrics

```
    """
```

Prepare data

```
X = df.drop(columns=[target_col])
```

```
y = df[target_col]
```

Encode categorical features

```
for col in X.columns:
```

```
    if X[col].dtype == 'object':
```

```
        le = LabelEncoder()
```

```
        X[col] = le.fit_transform(X[col].astype(str))
```

Encode target if categorical

```
if y.dtype == 'object':
```

```
    le = LabelEncoder()
```

```
    y = le.fit_transform(y)
```

```
    is_classification = True
```

```
else:
```

```
    is_classification = (y.nunique() < 20) # Heuristic for classification
```

Split data

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=test_size, random_state=random_state
```

)

Select model

if model_type == "Random Forest":

if is_classification:

model = RandomForestClassifier(n_estimators=100, random_state=random_state)

else:

model = RandomForestRegressor(n_estimators=100, random_state=random_state)

elif model_type == "Logistic Regression":

model = LogisticRegression(random_state=random_state, max_iter=1000)

elif model_type == "Decision Tree":

if is_classification:

model = DecisionTreeClassifier(random_state=random_state)

else:

model = DecisionTreeRegressor(random_state=random_state)

elif model_type == "XGBoost":

try:

import xgboost as xgb

if is_classification:

model = xgb.XGBClassifier(random_state=random_state)

else:

model = xgb.XGBRegressor(random_state=random_state)

except ImportError:

st.error("XGBoost not installed. Please install: pip install xgboost")

return None, None

else:

raise ValueError(f"Unknown model type: {model_type}")

Train model

model.fit(X_train, y_train)

Evaluate

y_pred = model.predict(X_test)

if is_classification:

metrics = {

'accuracy': accuracy_score(y_test, y_pred),

'precision': precision_score(y_test, y_pred, average='weighted', zero_division=0),

'recall': recall_score(y_test, y_pred, average='weighted', zero_division=0),

'f1': f1_score(y_test, y_pred, average='weighted', zero_division=0)

```

    }
else:
    metrics = {
        'r2': r2_score(y_test, y_pred),
        'mse': mean_squared_error(y_test, y_pred),
        'rmse': np.sqrt(mean_squared_error(y_test, y_pred))
    }

return model, metrics

```

Src/[utils.py](#)

```

"""
Utility Functions
"""

import pandas as pd
import streamlit as st
from pathlib import Path

@st.cache_data
def load_data(file_path):
    """Load data with caching"""
    try:
        df = pd.read_csv(file_path)
        return df
    except Exception as e:
        st.error(f"Error loading data: {e}")
        return None

def get_available_models():
    """Get list of available models"""
    return [
        "Random Forest",
        "Logistic Regression",
        "Decision Tree",
        "XGBoost"
    ]

def validate_dataframe(df):
    """Validate dataframe for analysis"""
    if df is None:
        return False, "No dataframe provided"

```

```

if df.empty:
    return False, "Dataframe is empty"

if len(df.columns) < 2:
    return False, "Dataframe needs at least 2 columns"

return True, "Valid dataframe"

def get_feature_types(df):
    """Get feature types"""
    numerical = df.select_dtypes(include=['float64', 'int64']).columns.tolist()
    categorical = df.select_dtypes(include=['object', 'category']).columns.tolist()
    datetime = df.select_dtypes(include=['datetime64']).columns.tolist()

    return {
        'numerical': numerical,
        'categorical': categorical,
        'datetime': datetime
    }

```

Src/[visualize.py](#)

```

"""
Visualization Functions
"""

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np

def plot_feature_importance(model, feature_names, top_n=10):
    """Plot feature importance"""
    if hasattr(model, 'feature_importances_'):
        importances = model.feature_importances_

        # Create DataFrame
        feat_imp = pd.DataFrame({
            'Feature': feature_names[:len(importances)],
            'Importance': importances
        }).sort_values('Importance', ascending=True)

        # Select top N

```

```

feat_imp = feat_imp.tail(top_n)

# Plot
fig, ax = plt.subplots(figsize=(10, 6))
ax.barh(feat_imp['Feature'], feat_imp['Importance'], color='steelblue')
ax.set_xlabel('Importance')
ax.set_title(f'Top {top_n} Feature Importances')
ax.grid(axis='x', alpha=0.3)

return fig
else:
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.text(0.5, 0.5, 'Feature importance not available\nfor this model type',
           ha='center', va='center', fontsize=12)
    ax.set_axis_off()
    return fig

def plot_confusion_matrix(y_true, y_pred, labels=None):
    """Plot confusion matrix"""
    from sklearn.metrics import confusion_matrix

    cm = confusion_matrix(y_true, y_pred)

    fig, ax = plt.subplots(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax)
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')
    ax.set_title('Confusion Matrix')

    return fig

def plot_learning_curve(train_scores, val_scores, title="Learning Curve"):
    """Plot learning curve"""
    fig, ax = plt.subplots(figsize=(8, 5))

    ax.plot(train_scores, label='Training Score', color='blue')
    ax.plot(val_scores, label='Validation Score', color='red')
    ax.set_xlabel('Training Iterations')
    ax.set_ylabel('Score')
    ax.set_title(title)
    ax.legend()
    ax.grid(alpha=0.3)

    return fig

```


Configuration file

```
[theme]
primaryColor = "#1f77b4"
backgroundColor = "#ffffff"
secondaryBackgroundColor = "#f0f2f6"
textColor = "#262730"
font = "sans serif"
```

```
[server]
maxUploadSize = 200
enableXsrfProtection = true
enableCORS = false
```

```
[browser]
gatherUsageStats = false
```

Requirements.txt

```
streamlit>=1.28.0
pandas>=2.0.0
numpy>=1.24.0
matplotlib>=3.7.0
seaborn>=0.12.0
plotly>=5.14.0
scikit-learn>=1.3.0
xgboost>=1.7.0
joblib>=1.3.0
openpyxl>=3.1.0
```

.gitignore

```
# Python
__pycache__/
*.py[cod]
*$py.class
*.so
.Python
```

env/
venv/
ENV/
build/
develop-eggs/
dist/
downloads/
eggs/
.eggs/
lib/
lib64/
parts/
sdist/
var/
wheels/
*.egg-info/
.installed.cfg
*.egg

Streamlit
.streamlit/cache/
.streamlit/secrets.toml

Data
*.csv
*.pkl
*.joblib
!data/sample_data.csv
!models/sample_model.pkl

IDE
.vscode/
.idea/
*.swp
*.swo
*~

OS
.DS_Store
[Thumbs.db](#)

[Readme.md](#)

ML Analytics Platform

A comprehensive **Streamlit application** for exploratory data analysis and machine learning predictions.

![Streamlit App](https://img.shields.io/badge/Streamlit-1.28.0-red)

![Python](https://img.shields.io/badge/Python-3.9+-blue)

![License](https://img.shields.io/badge/License-MIT-green)

Features

- **Exploratory Data Analysis**

- Missing value detection
- Statistical summaries
- Interactive visualizations
- Correlation analysis
- Outlier detection

- **Model Predictions**

- Multiple model support
- Real-time predictions
- Confidence scores
- Feature importance

- **Model Training**

- Train on uploaded data
- Multiple algorithms
- Performance metrics
- Download trained models

Installation

1. Clone the repository:

```
```bash
```

```
git clone https://github.com/yourusername/streamlit-ml-app.git
```

```
cd streamlit-ml-app
```